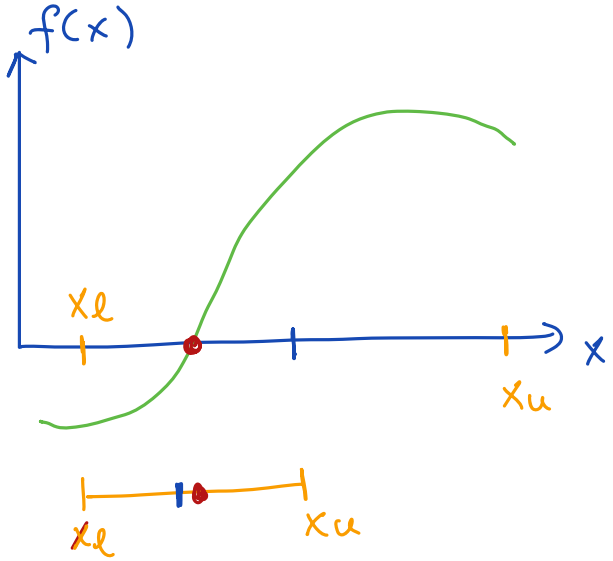
Chapter 6 – Roots: Open Methods

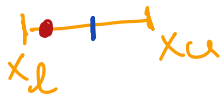
⇒ Bracketing methods are <u>always convergent</u>.

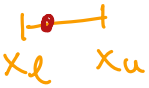
⇒ Open methods:

- require a single starting pt.
- use two starting which may not bracket the root

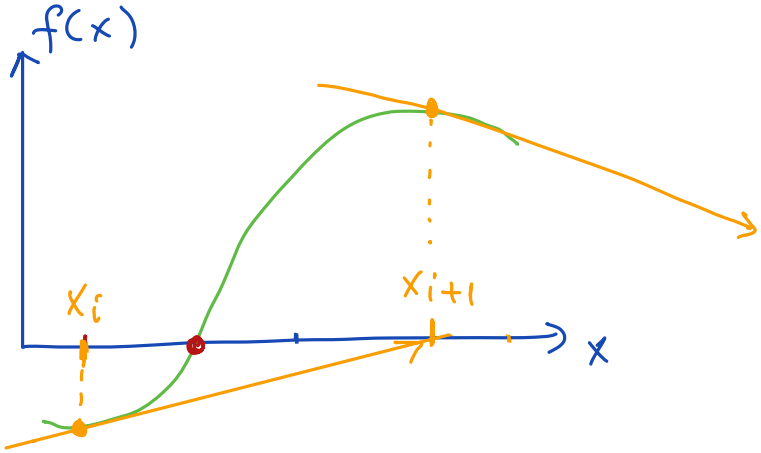
Bracketing Methods



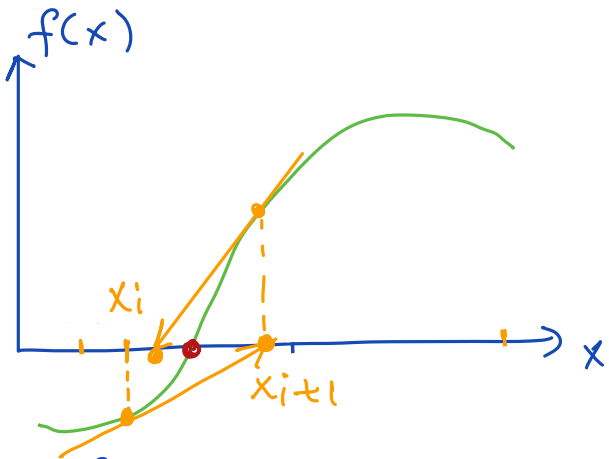




Open Methods



Diverges



Converger

We need a formula to go from x_i to x_{i+1} in an iterative way.

Simple Fixed Point Iteration (one-pt iteration, successive substitution)

- Arrange the function $f(x)=0$ so that x is on the left hand side of the eq^n. (by algebraic manipulation, by adding x to both sider) $x = g(x)$ (1)

EX $x^2-2x+3=0 \Rightarrow x = \frac{x^2+3}{2}$

$\sin x = 0 \Rightarrow x = \sin x + x$

- Write an iteration formula:

$$x_{i+1} = g(x_i)$$

- Eq (1) gives us a way predicting x based on function of an old value

Approximate error: $\varepsilon_a = \left| \frac{x_{i+1}-x_i}{x_{i+1}} \right| 100\%$

EX Locate the root of $f(x) = e^{-x} - x$ (using fixed-pt iteration)

$$x = e^{-x} \Rightarrow x_{i+1} = e^{-x_i}$$

Starting from $x_0 = 0$ (True root : 0.5671432g)

after 10 iterations $x_i = 0.564879$, $\varepsilon_a = 1.11\%$, $\varepsilon_t = 0.399\%$

Alternative Graphical Approach

Seperate the eq^n into two components:

$$f_1(x) = f_2(x)$$

Then $y_1 = f_1(x)$ and

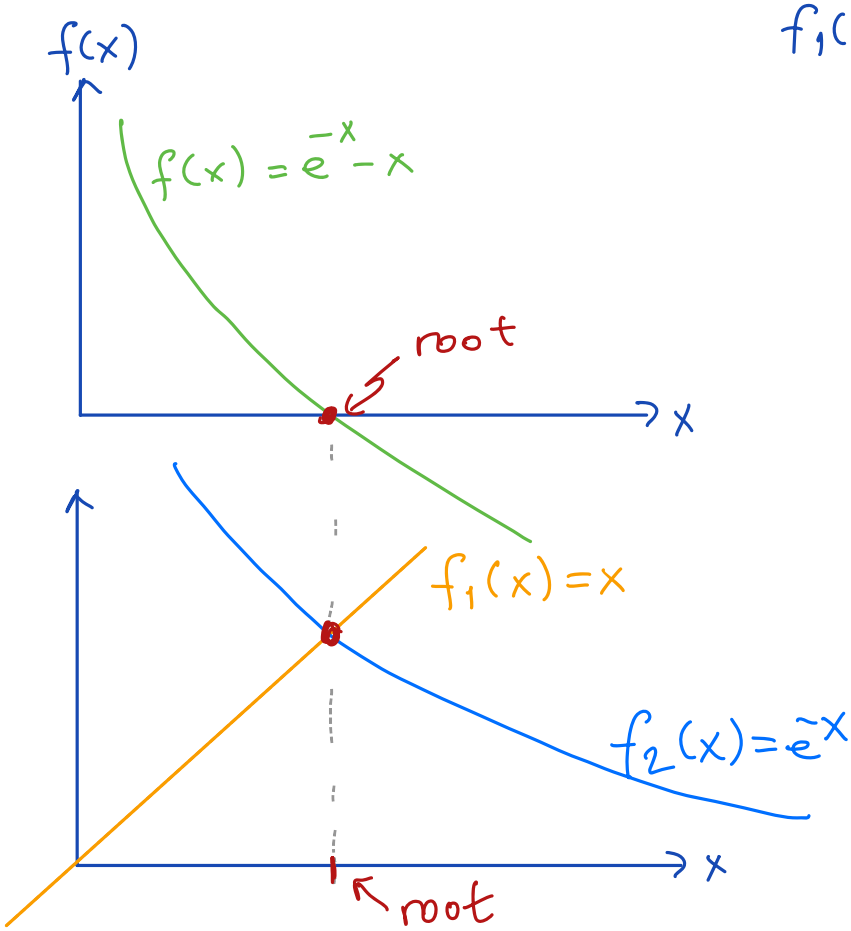
$y_2 = f_2(x)$ can be plotted seperately.

The x values corresponding to the intersection of y_1 and y_2 represent the roots $f(x)=0$

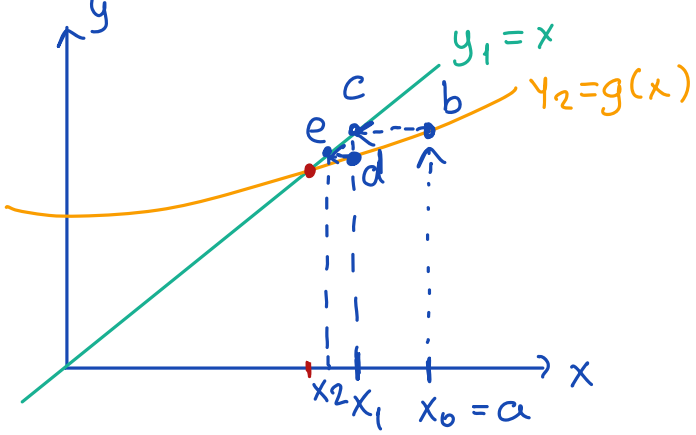
EX $e^{-x} - x = 0$ Determine the roots graphically

Reformulate as:

$y_1 = x$ and $y_2 = e^{-x}$ $\Rightarrow$ $\underbrace{x}_{f_1(x)} = \underbrace{e^{-x}}_{f_2(x)}$



Two-curve Method to Illustrate the Convergence and Divergence of Fixed-Pt iteration.



$\underbrace{y_1 = f_1(x) = f_2(x) = y_2}_{x = g(x)}$

convergent

a → initial guess x_0

b → determine the corresponding pt on y_2 curve $(x_0, g(x_0))$

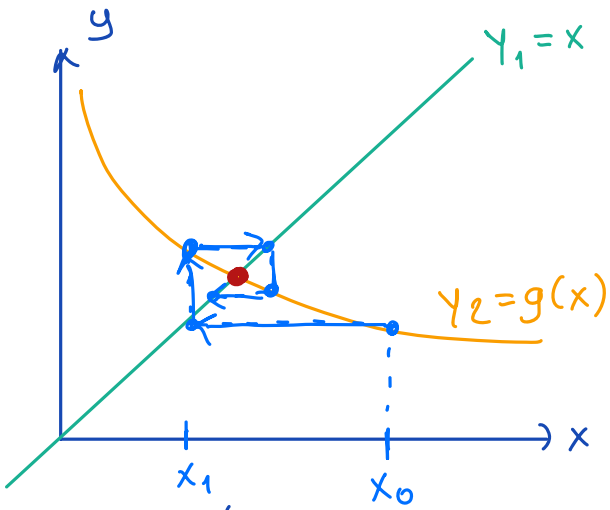
c,d → locate $(x_1, g(x_1))$ by moving left horizontally to y_1 curve

a,b,c,d are equivalent to 1st iteration in fixed-pt method

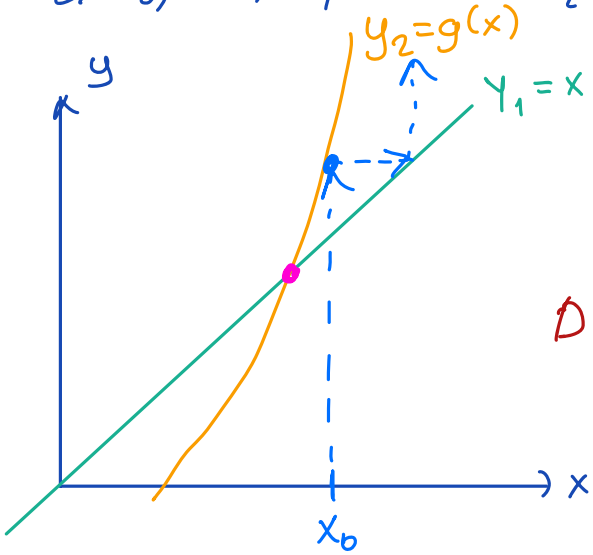
$x_1 = g(x_0)$

Next iteration: move to $(x_1, g(x_1))$ (d)

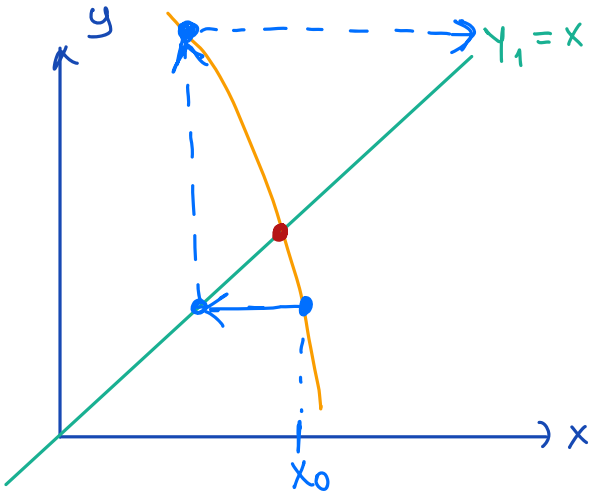
move to (x_2, x_2) (e), where $x_2 = g(x_1)$



Convergent



Diverge



Diverge

Converges if

$$|g'(x)| < 1$$ (Derivation in the book)

Derivative near the root

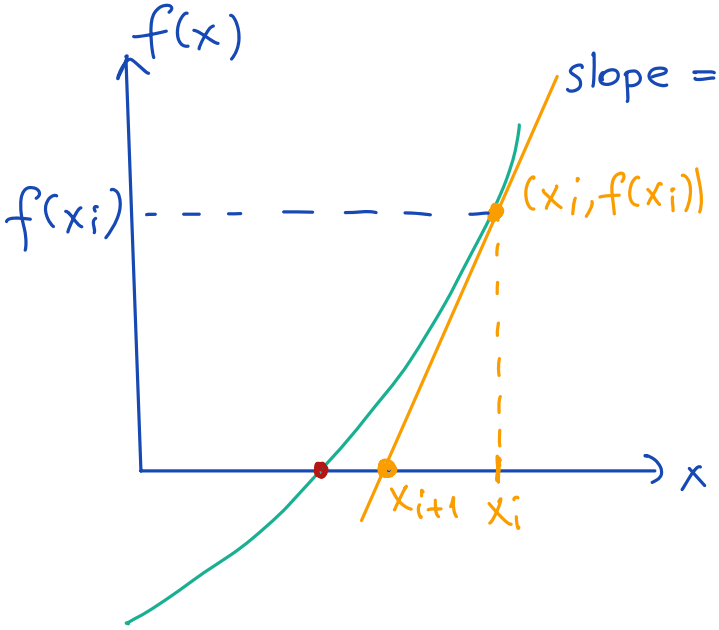
The Newton-Raphson Method

→ The most widely wed root-locating method.

→ initial guess: x_i

→ Extend a tangent from the pt $(x_i, f(x_i))$

→ Point where the tangent crosses the x-axis is an improved root estimate



Newton-Raphson

$$f'(x_i) = \frac{f(x_i) - 0}{x_i - x_{i+1}}$$

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

Newton-Raphson Formula

EX Estimate the root of $e^{-x} - x$, initial guess: $x_0 = 0$

$$f'(x) = -e^{-x} - 1$$

$$x_{i+1} = x_i - \frac{e^{-x_i} - x_i}{-e^{-x_i} - 1}$$

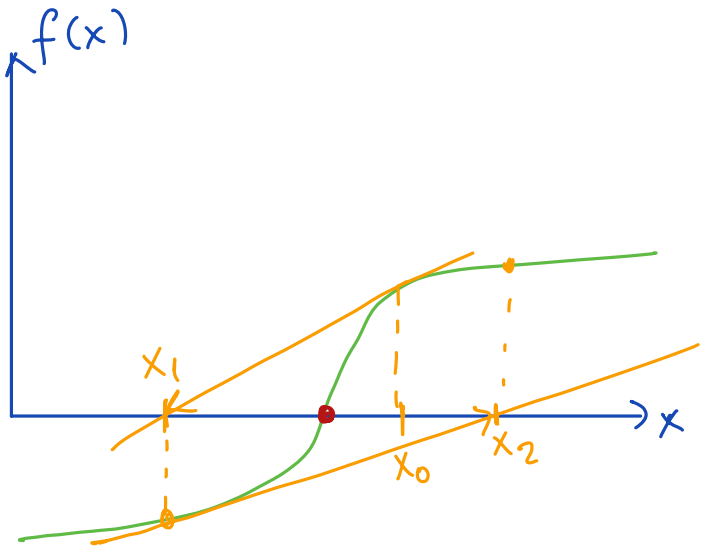
iteration	x	ε_t (%)
0	0	100
1	0,5	11.8
2	0,5663	0.147
3	0.5671	0.00002
4	0.567143	$< 10^{-8}$

Converges quickly

Problems of Newton-Raphson Method

- There's no general convergence criterion
- Convergence depends on the nature of the function and on the accuracy of initial guess.

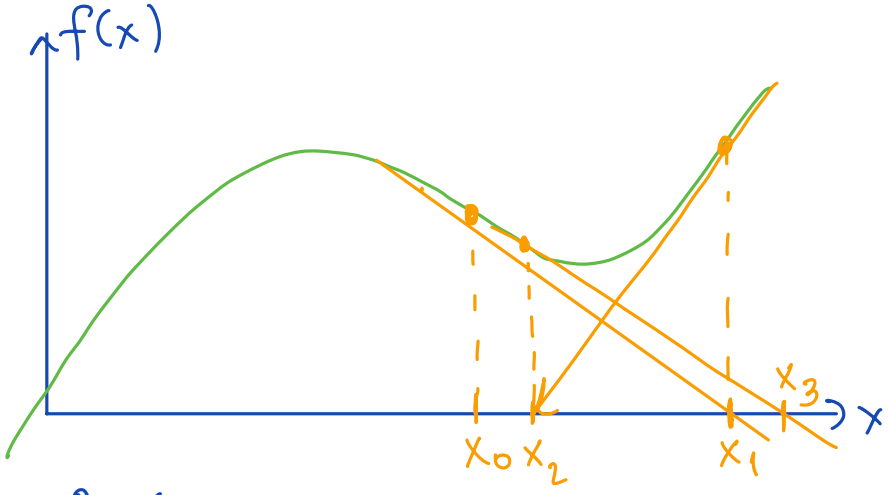
Problem 1



Inflection pt in the vicinity of the root.
($f''(x)=0$)

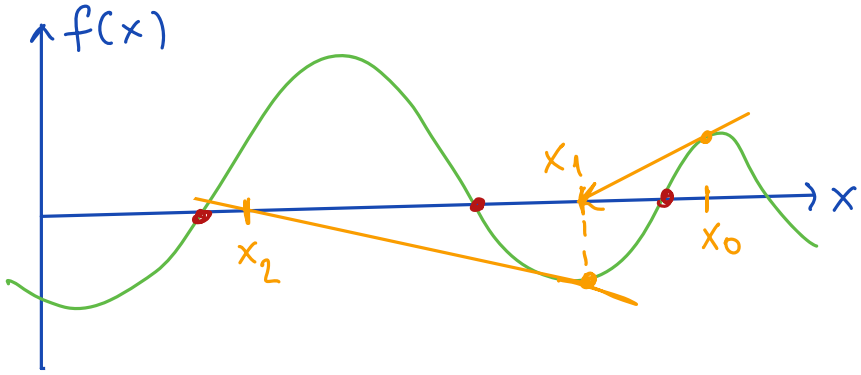
cause of divergence

Problem 2



Oscillation around a local min.

Problem 3



Initial guess is close to one root can jump to several roots away. (because of near-zero slopes)

$\Rightarrow f'(x_i) = 0$ is a disaster (division by zero)

(graphically: tangent never crosses the x-axis.

The Secant Method

→ In Newton-Raphson derivative may be a problem.

→ use backward finite difference method of the derivative.

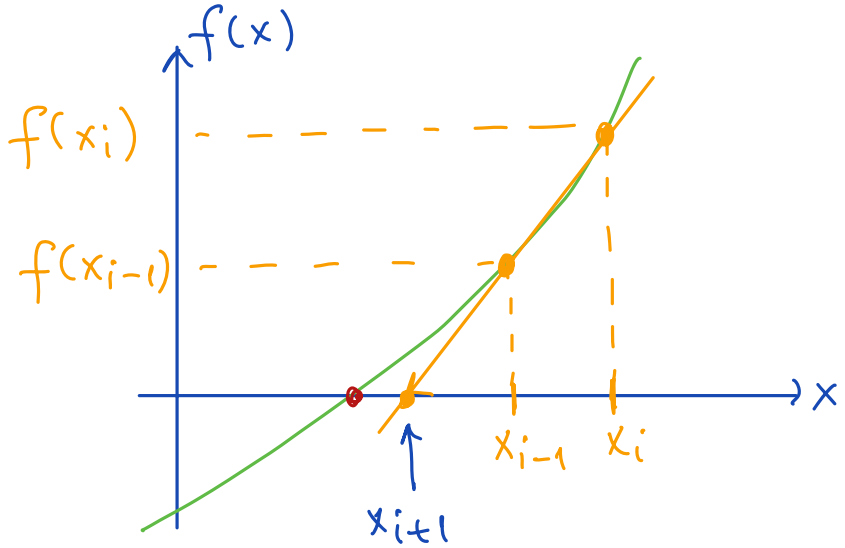
$$f'(x_i) \cong \frac{f(x_{i-1}) - f(x_i)}{x_{i-1} - x_i}$$

↳ substitute into Newton-Raphson

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

$$x_{i+1} = x_i - \frac{f(x_i)\,(x_{i-1} - x_i)}{f(x_{i-1}) - f(x_i)}$$

Secant Method.



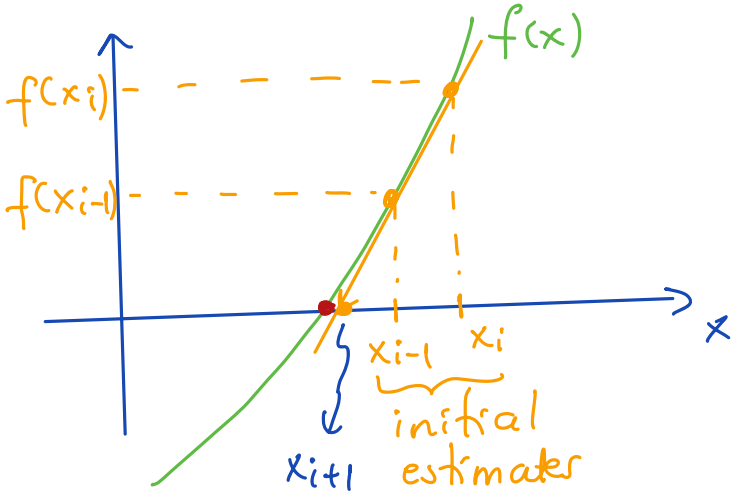
Secant Method

user a difference rather than a derivative

Difference between Secant Method and False-Position Methods

Secant Method

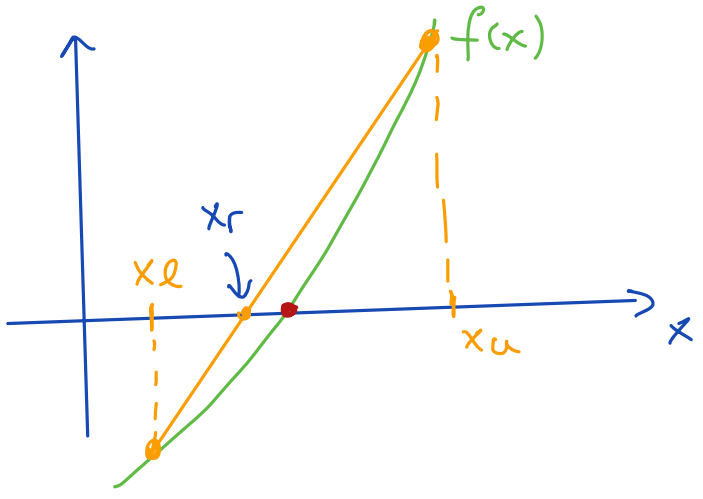
$$x_{i+1} = x_i - \frac{f(x_i)(x_{i-1} - x_i)}{f(x_{i-1}) - f(x_i)}$$



May diverge since root is not bracketed.

False-Position

$$x_r = x_u - \frac{f(x_u)(x_l - x_u)}{f(x_l) - f(x_u)}$$



always converges

$x_l \leftarrow x_r$

Two estimates always bracket the root.

Difference: how one of the initial value is updated.

Ex Find the root of $f(x) = e^{-x} - x$. Start with $x_{-1} = 0$ and $x_0 = 1$

True root: 0.56714329...

First iteration: $x_{-1} = 0$, $f(x_{-1}) = 1$

$x_0 = 1$, $f(x_0) \simeq -0.63212$

$$x_{i+1} = x_i - \frac{f(x_i)(x_{i-1} - x_i)}{f(x_{i-1}) - f(x_i)}$$

$$x_1 = x_0 - \frac{f(x_0)(x_{-1} - x_0)}{f(x_{-1}) - f(x_0)} = 1 - \frac{(-0.63212)(0-1)}{1 - (-0.63212)} = 0.61270$$

$\varepsilon_t = 8\%$

2nd iter: $x_0 = 1$, $f(x_0) = -0.63212$

$x_1 = 0.61270$, $f(x_1) = -0.07081$

Both estimates are on the same side of the root.

$$x_2 = x_1 - \frac{f(x_1)(x_0 - x_1)}{f(x_0) - f(x_1)} = 0.56384 \Rightarrow \varepsilon_t = 0.58\%$$

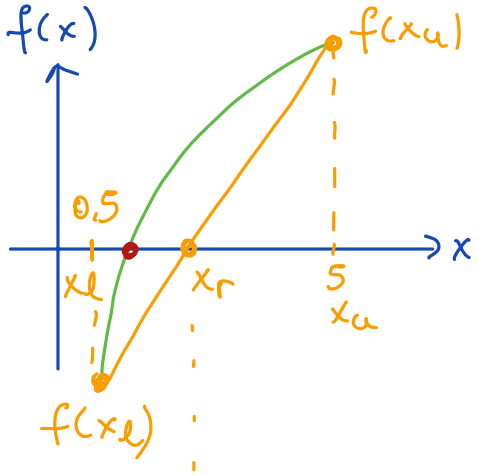
3rd iter $x_3 = 0.56717$, $\varepsilon_t = 0.0048\%$

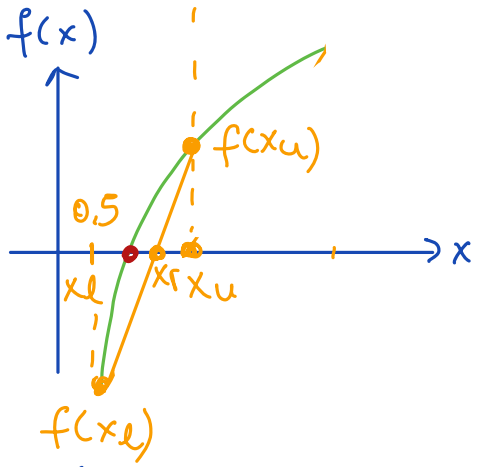
EX Comparison of Convergence of Secant and false-Position method

Estimate the root of $f(x) = \ln x$

Start with $x_l = x_{i-1} = 0.5$ and $x_u = x_i = 5$

False Position

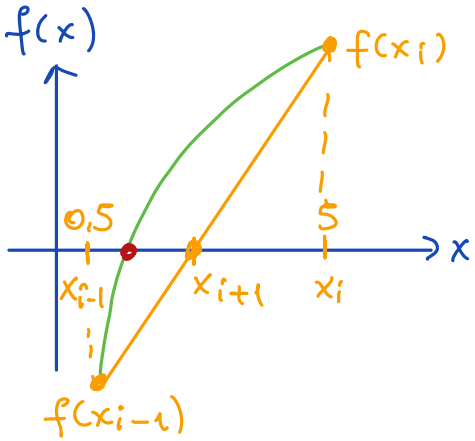


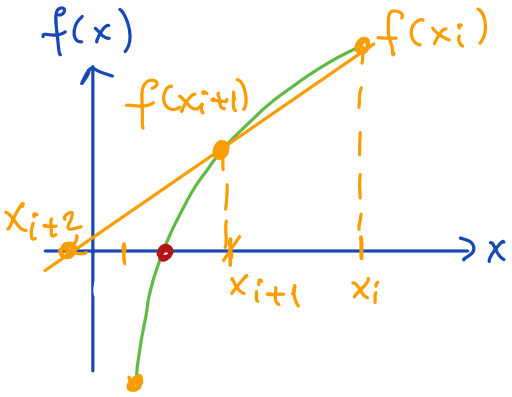


iteration 2 : $x_r = 1.0585$

converging

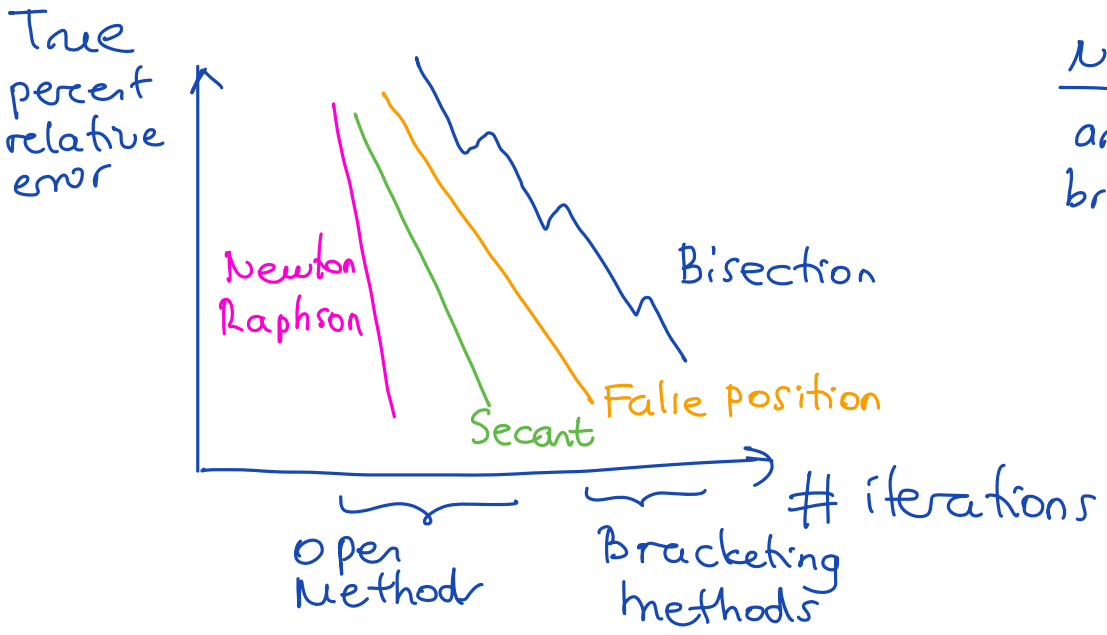
Secant





iteration 2 : $x_r = x_{i+2} = -0.10438$

diverging.



Note: open methods are faster than bracketing methods

Modified Secant Method

Rather than using two arbitrary values to estimate the derivative, use a fractional perturbation

$$f'(x_i) \cong \frac{f(x_i+\delta x_i) - f(x_i)}{\delta x_i}$$, δ : a small perturbation fraction

$$x_{i+1} = x_i - \frac{f(x_i)\,\delta x_i}{f(x_i+\delta x_i) - f(x_i)}$$

Brent's method

→ Hybrid method combining reliability of bracketing with the speed of open methods.

→ Applies open methods (speedy) whenever possible

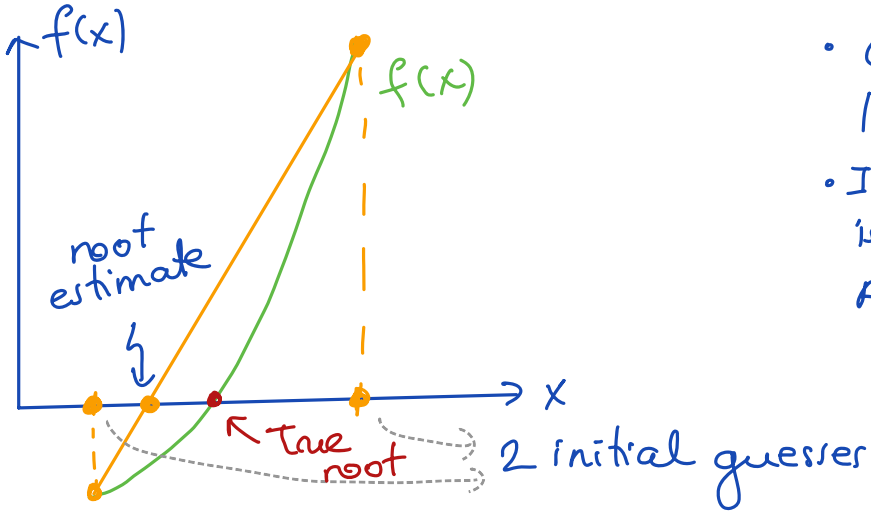
→ Revert to a reliable bracketing method if necessary.

→ Bracketing: bisection

→ open methods: secant and inverse quadratic interpolation.

Inverse Quadratic Interpolation

Similar to secant in spirit

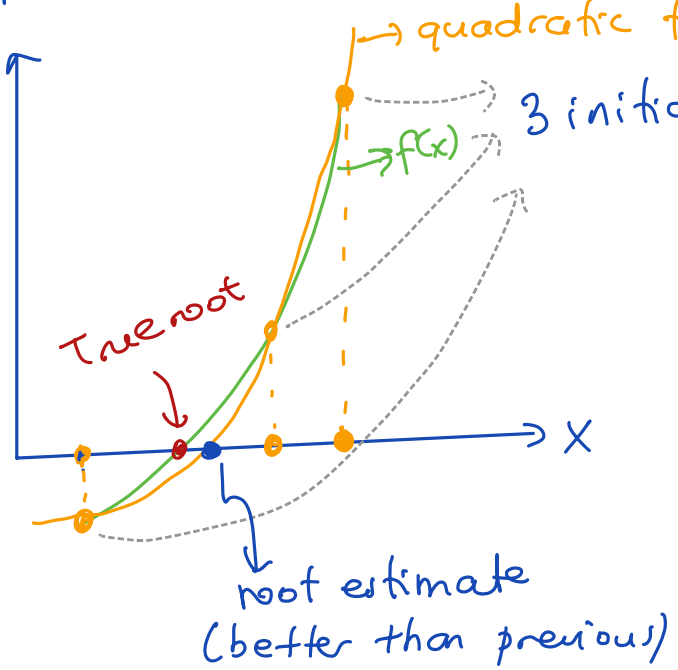


- Compute a straight line that thru 2 guesser
- Intersection with the x-axis is the root estimate

Also called as:

linear interpolation

Suppose we had 3 initial guesser.



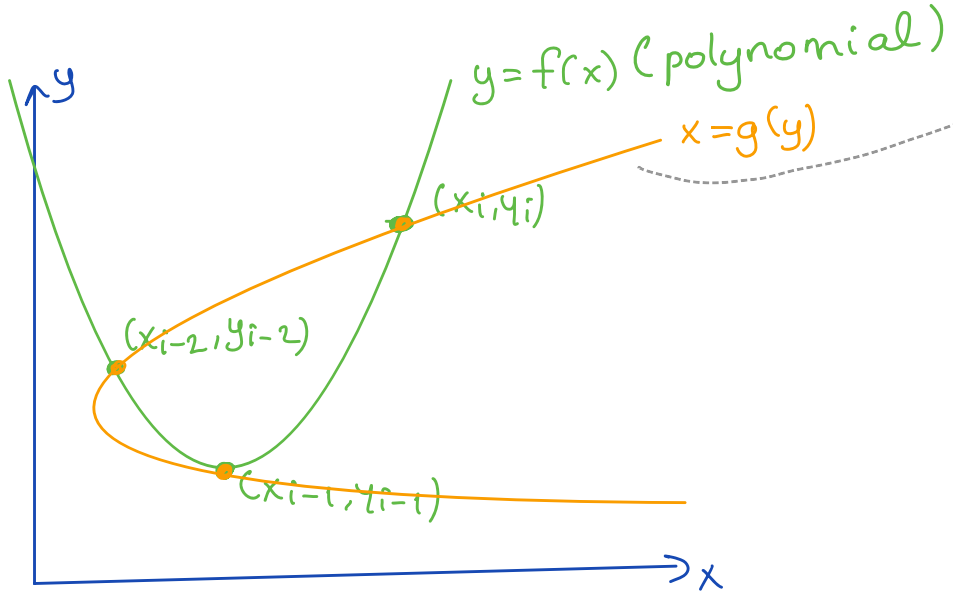
⇒ Fit a quadratic function

⇒ intersection with the x-axis is the root estimate

⇒ using a curve rather than a line gives a better estimate

Problem : The parabola may not intersect the x-axis!!
(happens when the roots are complex)

Solution : use inverse quadratic interpolation (create a sideways parapola → always intersects the x-axis)



A sideways parabola always intersects the x-axis.

Three points (initial guesses) (x_{i-2}, y_{i-2}), (x_{i-1}, y_{i-1}), (x_i, y_i)

A quadratic function of y: (use Lagrange polynomial)

$$g(y) = \frac{(y-y_{i-1})(y-y_i)}{(y_{i-2}-y_{i-1})(y_{i-2}-y_i)} x_{i-2} + \frac{(y-y_{i-2})(y-y_i)}{(y_{i-1}-y_{i-2})(y_{i-1}-y_i)} x_{i-1} + \frac{(y-y_{i-2})(y-y_{i-1})}{(y_i-y_{i-2})(y_i-y_{i-1})} x_i$$

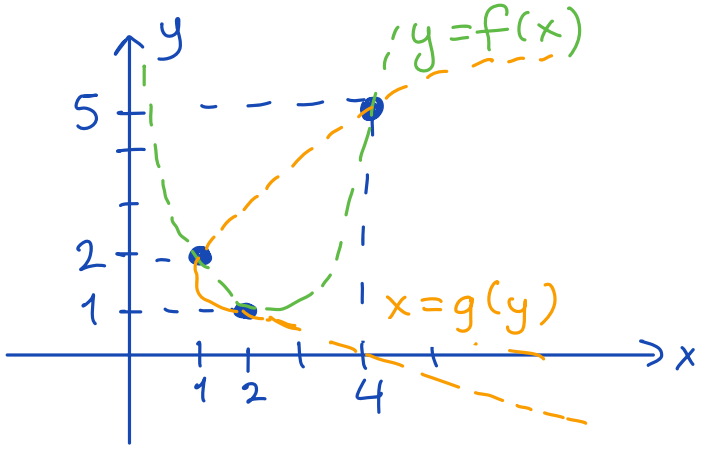
The root is x_{i+1} corresponds to $y=0$

$$g(y) = \frac{y_{i-1}\, y_i}{(y_{i-2}-y_{i-1})(y_{i-2}-y_i)} x_{i-2} + \frac{y_{i-2}\, y_i}{(y_{i-1}-y_{i-2})(y_{i-1}-y_i)} x_{i-1} + \frac{y_{i-2}\, y_{i-1}}{(y_i-y_{i-2})(y_i-y_{i-1})} x_i$$

Ex Develop quadratic equations in both x and y for the data pts: (1,2), (2,1) and (4,5)

(x_{i-2}, y_{i-2}) (x_{i-1}, y_{i-1}) (x_i, y_i)

$y=f(x)$ and $x=g(y)$



(1,2), (2,1), (4,5)

$$f(x) = \frac{(x-2)(x-4)}{(1-2)(1-4)}2 + \frac{(x-1)(x-4)}{(2-1)(2-4)}1 + \frac{(x-1)(x-2)}{(4-1)(4-2)}5$$

$f(x) = x^2 - 4x + 5 \implies$ roots : $2 \pm i$ (complex)

$$g(y) = \frac{(y-1)(y-5)}{(2-1)(2-5)}1 + \frac{(y-2)(y-5)}{(1-2)(1-5)}2 + \frac{(y-2)(y-1)}{(5-2)(5-1)}4$$

$= 0.5y^2 - 2.5y + 4$ (no real roots in y)

put $y = 0$

$g(y)\Big|_{y=0} = 4 = x_{i+1}$

MATLAB Functions

fzero: ⇒ fzero(function, x0)
- x0 → initial guess
- function → name of function being evaluated

⇒ fzero(function, [x0 x1])
- [x0 x1] → two initial guesses

```
>> x = fzero(@(x) x^2-9, -4)
```
finds the negative root (has 2 roots at ± 3)

x = -3 → output

```
>> x = fzero(@(x)(x^2-9), 4)
```
4 → guess; finds the positive root

x = 3 (output)

Note: fzero works as follows:
- if a single initial guess is passed, it performs a search to identify a sign change (takes increasingly bigger steps in both directions until a sign change is detected)
- then uses fast method or bisection

EX: to display iterations

```
>> options = optimset('display','iter');
>> [x,fx] = fzero(@(x) x^10-1, 0.5, options)
```
(35 iterations will be displayed)

x = 1 , fx = 0 → outputs

EX to use a less stringent tolerance

```
>> options = optimset('tolx', 1e-3);
>> [x,fx] = fzero(@(x) x^10-1, 0.5, options);
```
x = 1.0009 fx = 0.0090

Polynomials

$$f_n(x) = a_1 x^n + a_2 x^{n-1} + \cdots + a_{n-1} x^2 + a_n x + a_{n+1}$$

roots function in MATLAB

```
>> x = roots(c)
```

- c → vector containing polynomial coefficients: $c = [a_1\ a_2 \cdots a_{n+1}]$
- x → a column vector containing roots

EX Let $f(x) = x^5 - 3.5x^4 + 2.75x^3 + 2.125x^2 - 3.875x + 1.25$

Roots: $0.5, -1, 2, 1 \pm 0.5i$

in MATLAB:

```
>> c = [1 -3.5 2.75 2.125 -3.875 1.25];
>> polyval(c,1) % evaluate polynomial at 1
>> ans = -0.25
>> x = 2.0
       1.0
       1 ± 0.5i
       0.5
>> a = poly(x)
```

Polynomial Evaluation

Let $f(x) = a_3 x^3 + a_2 x^2 + a_1 x + a_0$ (1)

Evaluating (1) for a given x requires: 6 multiplications
3 additions

A nested format

$$f(x) = ((a_3 x + a_2)x + a_1)x + a_0$$

requires: 3 multiplications, 3 additions

n^{th} order poly: n " , n "

nested format minimizes the # eq^{n}s, also minimizes round-off errors

Systems of Nonlinear Equations

Goal: Locate the roots of a set of simultaneous equations.

$$f_1(x_1, x_2, \ldots, x_n) = 0$$
$$\vdots \qquad\qquad \vdots$$
$$f_n(x_1, x_2, \ldots, x_n) = 0$$

Ex
$$u(x,y) = x^2 + xy - 10 = 0 \quad (1)$$
$$v(x,y) = y + 3xy^2 - 57 = 0 \quad (2)$$
} correct roots $x = 2$ and $y = 3$

Use fixed-pt iteration

initial guesses $x = 1.5$ and $y = 3.5$

From (1): $$x_{i+1} = \frac{10 - x_i^2}{y_i} \quad (3)$$

From (2): $$y_{i+1} = 57 - 3x_i y_i^2 \quad (4)$$

Use eqns (3) and (4) iteratively:

1st iterat.

from (3): $$x_1 = \frac{10 - (1.5)^2}{3.5} = 2.21429 \qquad (x_0 = 1.5,\ y_0 = 3.5)$$

from (4): $$y = 57 - 3(2.21429)(3.5)^2 = -24.37516$$

(y_0; diverging)

2nd iter.

from (3): $$x = \frac{10 - (2.21429)^2}{-24.375} = -0.209$$

from (4): $$y = 429.709$$

Solution: write the original update eqns differently:

$$u(x,y) = x^2 + xy - 10 = 0 \qquad (1)$$

$$v(x,y) = y + 3xy^2 - 57 = 0 \qquad (2)$$

From (1): $x_{i+1} = \sqrt{10 - x_i y_i}$ (3)

From (2): $y_{i+1} = \sqrt{\frac{57 - y_i}{3x_i}}$ (4)

1st iteration: $x = 2.17$ $y = 2.86$

2nd " : $x = 1.94$ $y = 3.04$

converging to the true values $x = 2$ and $y = 3$

Sufficient conditions for convergence are:

$$\left|\frac{\partial u}{\partial x}\right| + \left|\frac{\partial u}{\partial y}\right| < 1$$

and

$$\left|\frac{\partial v}{\partial x}\right| + \left|\frac{\partial v}{\partial y}\right| < 1$$

very restrictive ⇒ fixed-pt iteration has limited utility for solving nonlinear systems.

Newton-Raphson (for Systems of Nonlinear Eq^{n}s)

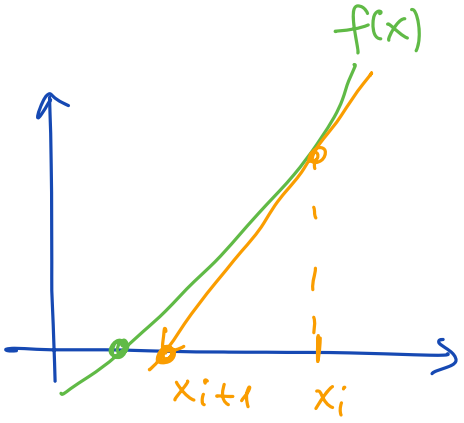
single eq^n $f(x) = 0$

1st order Taylor series

$$f(x_{i+1}) = f(x_i) + (x_{i+1} - x_i) f'(x_i) = 0$$

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

Single eq^n form of Newton-Raphson



Two Equations

$$\left.\begin{aligned} u(x,y) &= 0 \\ v(x,y) &= 0 \end{aligned}\right\}$$

$u(x_i, y_i) \Rightarrow u_i \qquad u(x_{i+1}, y_{i+1}) = u_{i+1}$

$$\begin{cases} u_{i+1} = u_i + (x_{i+1} - x_i)\dfrac{\partial u_i}{\partial x} + (y_{i+1} - y_i)\dfrac{\partial u_i}{\partial y} + \cancel{H.O.T.} = 0 \\ v_{i+1} = v_i + (x_{i+1} - x_i)\dfrac{\partial v_i}{\partial x} + (y_{i+1} - y_i)\dfrac{\partial v_i}{\partial y} + \cancel{H.O.T} = 0 \end{cases}$$

Solve the 2 eqn above

$$\begin{cases} x_{i+1} = x_i - \dfrac{u_i \dfrac{\partial v_i}{\partial y} - v_i \dfrac{\partial u_i}{\partial y}}{\dfrac{\partial u_i}{\partial x}\dfrac{\partial v_i}{\partial y} - \dfrac{\partial u_i}{\partial y}\dfrac{\partial v_i}{\partial x}} \\ y_{i+1} = y_i - \dfrac{v_i \dfrac{\partial u_i}{\partial x} - u_i \dfrac{\partial v_i}{\partial x}}{\dfrac{\partial u_i}{\partial x}\dfrac{\partial v_i}{\partial y} - \dfrac{\partial u_i}{\partial y}\dfrac{\partial v_i}{\partial x}} \end{cases}$$